



Report

RC4

Name /Mohammed Talal Alobaysi

RC4: A Stream Cipher Overview

RC4 (Rivest Cipher 4) is a widely known stream cipher developed by **Ron Rivest** in 1987 for RSA Security. It gained popularity due to its simplicity, speed, and ease of implementation in both software and hardware environments.

Key Features

- **Type:** Stream cipher
- **Key Length:** Variable, typically between 40 and 2048 bits
- **Operation:** Byte-oriented
- **Algorithm Components:**
 - **Key Scheduling Algorithm (KSA):** Initializes the permutation of a 256-byte state array (S) based on the key.
 - **Pseudo-Random Generation Algorithm (PRGA):** Produces a key stream byte-by-byte, which is XORed with plaintext to produce ciphertext (and vice versa for decryption).

Historical Use

RC4 was widely used in protocols such as **SSL/TLS (up to version 1.2)**, **WEP (Wired Equivalent Privacy)**, and **Wi-Fi Protected Access (WPA)** due to its efficiency and ease of implementation. It was particularly attractive for performance-critical applications on limited-resource devices.

Security Concerns

Despite its initial popularity, RC4 has been **deprecated in modern cryptographic applications** due to multiple vulnerabilities:

- **Key scheduling flaws:** Weaknesses in the KSA can lead to key leakage and predictable outputs, especially in implementations like WEP.
- **Biased output:** The PRGA generates non-uniform output, making statistical attacks viable.
- **State recovery attacks:** Given sufficient output, attackers can reconstruct parts of the internal state and potentially recover the key.

As a result, RC4 is now considered insecure for use in cryptographic systems. Organizations such as the **IETF** and **Microsoft** have formally advised against its use, and modern protocols have transitioned to more robust ciphers like **AES (Advanced Encryption Standard)**.

RC4: A Stream Cipher Overview with Examples

RC4 (Rivest Cipher 4) is a symmetric key stream cipher designed by **Ron Rivest** in 1987. It was widely used due to its simplicity and speed but is now considered obsolete due to multiple vulnerabilities.

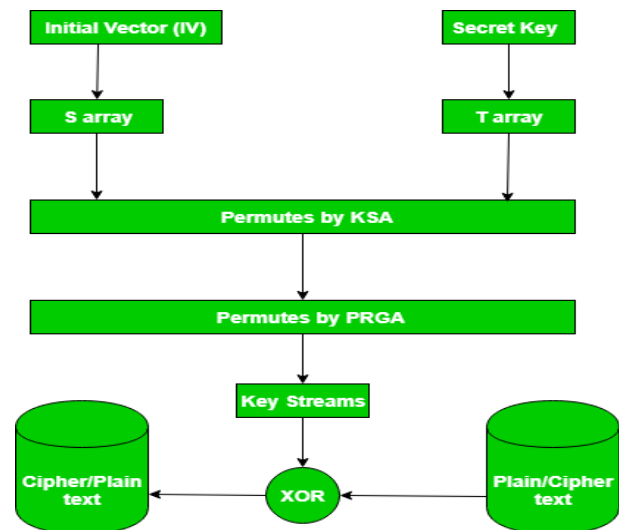
1. Core Components

a. Key Scheduling Algorithm (KSA)

The KSA initializes a 256-byte state array *S* based on the input key. It performs a series of permutations to mix the key into the state array.

b. Pseudo-Random Generation Algorithm (PRGA)

After the KSA, the PRGA generates a keystream by continually shuffling the state array and selecting bytes from it. The keystream is then XORed with the plaintext to produce the ciphertext.



2. Example of Encryption and Decryption with RC4

Let's consider a **simple example** using a short key and plaintext for demonstration.

Input:

- **Key:** "Key"
- **Plaintext:** "Plain"

Step 1: Key Scheduling Algorithm (KSA)

The key is expanded and used to initialize the state array *S*. For this short key, the internal permutation of *S* changes based on the ASCII values of "K", "e", and "y".

Step 2: Pseudo-Random Generation Algorithm (PRGA)

After initialization, the PRGA generates a keystream. Suppose the keystream generated for the key "Key" is (in hexadecimal):

EB 9F 77 81 B7

Step 3: XOR Keystream with Plaintext

Plaintext in ASCII:

P l a i n

50 6C 61 69 6E (hex)

XOR with keystream:

$$50 \oplus EB = BB$$

$$6C \oplus 9F = F3$$

$$61 \oplus 77 = 16$$

$$69 \oplus 81 = E8$$

$$6E \oplus B7 = D9$$

Ciphertext (in hexadecimal):

BB F3 16 E8 D9

Step 4: Decryption

To decrypt, apply the **same keystream** to the ciphertext:

$$BB \oplus EB = 50$$

$$F3 \oplus 9F = 6C$$

$$16 \oplus 77 = 61$$

$$E8 \oplus 81 = 69$$

$$D9 \oplus B7 = 6E$$

Resulting in the original plaintext:

50 6C 61 69 6E → "Plain"

3. Applications and Security Issues

Historical Uses:

- **WEP (Wired Equivalent Privacy):** Used in early Wi-Fi networks, but easily broken due to RC4 flaws.
- **TLS (Transport Layer Security):** Supported RC4 in earlier versions, now deprecated.

Security Flaws:

- **Key biases:** Certain key patterns lead to predictable keystream output.
- **Initialization weaknesses:** Early bytes of the keystream are not secure and can leak information.
- **Practical attacks:** Tools exist that exploit RC4 weaknesses to recover plaintext from encrypted traffic, especially in WEP.

Conclusion: RC4 played a significant role in the development of stream ciphers and the early adoption of secure communication protocols. However, due to its inherent vulnerabilities, it has been **phased out** in favor of more secure encryption methods. Understanding RC4's design and limitations remains important from a cryptographic research and historical perspective.